CDT Healthcare Data Science

ML for Time Series Module

Wearable Laboratory 1: From ECG Signals to Features and a Classifier

Organiser(s): ..... [Organiser to be confirmed]



Department of Engineering Science

University of Oxford

Nov 2026

Table of contents

0 Lab overview 1
1 What is an ECG signal? 2
2 What do people measure from an ECG? The rhythm 5
3 Can we extract them ourselves? The recipe 8
4 Finding the beats: detector v2, which denoises, seeks peaks and imputes 11
4.1 A reference and a score 11
4.2 Peak finder, step 1: denoise with a band-pass filter 13
4.3 Peak finder, step 2: differentiate, square, integrate 14
4.4 Peak finder, step 3: locate the R peak on the ECG 15
4.5 Median imputation: correct the beats with the median RR 17
4.6 How close did we get? 18
5 What the features are for: classifying the rhythm 20
5.1 54 features for one recording 20
5.2 The feature table and the classifier 21
Submitting your work 23

Bibliography **24**

0 Lab overview

This lab takes a raw electrocardiogram (ECG) from a wearable device, turns it into a few numbers that describe the heart's rhythm, and uses such numbers to classify the rhythm. It is organised around five questions:

Question	Your work	Notebook
1 What is an ECG signal?	Label the P, Q, R, S and T dots on five excerpts	1
2 What do people measure from it?	Code heart rate, SDRR and RMSSD from NeuroKit2's RR sequences	2
3 Can we extract them ourselves?	The recipe: a peak finder and median imputation; code detector v1, the naive detector	3
4 Finding the beats	Code the four steps of detector v2 (filter, envelope, locate, impute), each scored against NeuroKit2	4, 4A–4D
5 What are the features for?	Code a RandomForest on the full 54 NeuroKit2 features, on Lab 2's split	5

Here the features are *designed by experts*. In Lab 2 the same recordings are classified by networks that learn their own features from the signal.

How to work. Open `ECG_Preprocessing.ipynb`. Run the supplied cells, complete the cells marked **YOUR CODE**; every task is a function, checked against a reference. Unfinished exercises print a reminder rather than stopping later cells. The notebook holds the full wording of every task; this note explains the ideas behind them.

Setup. The lab lives in a repository (https://github.com/junjiechen-chris/CDT-HDS-ECG-Lab-1). On Colab, open the notebook with the badge in its first cell and save a copy in your Drive; its Setup cell fetches the code, installs NeuroKit2 [1] and scikit-learn, downloads the data zip and asks for its password, which is given in the lab. Locally, clone the repository, install `requirements.txt` and `requirements-comparison.txt`, and place the data zip in `data/`. No GPU is needed. The data are pickle files, which can run code when loaded: the notebook enables its trust flag for the lab's own zip only.

1 What is an ECG signal?

Notebook: Section 1

In one sentence. An ECG is the voltage between two electrodes on the skin, sampled many times a second, and every heartbeat leaves the same short pattern in it: a P wave, a QRS complex and a T wave (Figure 1, top). The sharp R peak at the top of the QRS marks *when* the beat happened, and everything later in this lab is built from those times. In wearable recordings the pattern often sits on drift, noise or an inverted polarity (Figure 1, bottom), which is why the beats have to be found by a detector that survives all three.

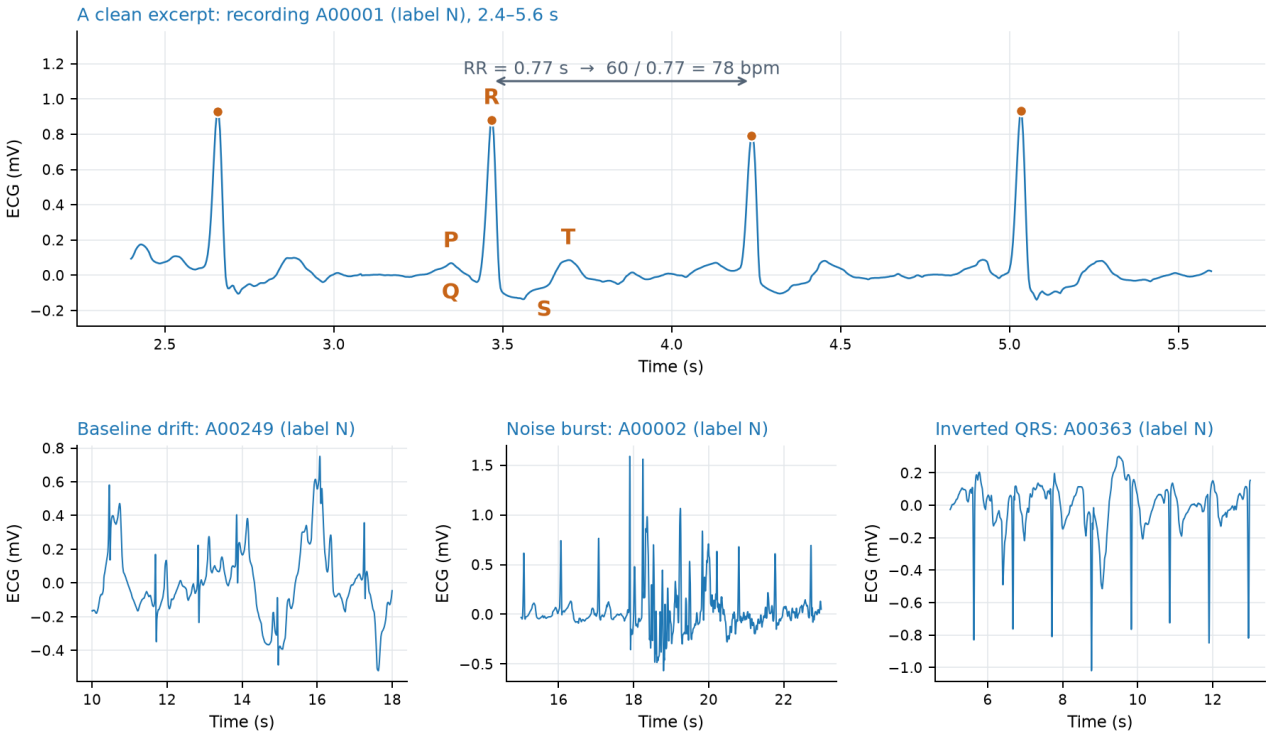


Figure 1: Top: three beats of a clean excerpt, with the P, Q, R, S and T waves of one beat and the RR interval between two R peaks. Bottom: three things that get in the way, from recordings that are all labelled `N`: baseline drift, a noise burst, and a QRS that points down.

The waves of one beat. The **P wave** is small and rounded: the atria contract. The **QRS complex** is a sharp spike about 80 ms to 120 ms wide and usually the tallest feature: the ventricles contract. Q and S are the small dips either side of R. The **T wave** is broad and rounded: the ventricles recover. The waves come in this order in every normal beat, so their positions relative to R are nearly fixed; the wave-shape features used in Section 5 measure exactly those positions and heights.

Timing. The two marked R peaks are 0.77 s apart. That is one **RR interval**, and $60/0.77 = 78$ beats per minute. At rest the intervals are about 0.6 s to 1.0 s (60 bpm to 100 bpm). The RR interval is the first feature of this lab, and you can read it straight off the plot.

Three things that get in the way. *Baseline drift* (bottom left) comes from breathing, movement and electrode contact: slow, below about 0.5 Hz, and the beats ride up and down on it, so "high" no longer means "beat". A *noise burst* (bottom middle) comes from movement or muscle: for a few seconds the spikes are as tall as the R peaks; this recording is still labelled `N`, because the label describes the rhythm, not the quality. An *inverted QRS* (bottom right) comes from where the electrodes sit: the QRS points down, so the maximum of the signal is never the beat. Each of these defeats a naive rule, and Section 4 has one step for each.

The recordings. They come from the PhysioNet/Computing in Cardiology Challenge 2017 [2, 3], educational use only.

Item	Value
Device	a handheld single-lead recorder, two electrodes under the fingers
Recordings	2,000: `N` 1,187, `A` 196, `O` 553, `~` 64
Length	9 to 60 seconds
Sampling rate	300 Hz: time in seconds = sample index / 300
Units	stored in microvolts; the notebook divides by 1,000 to plot millivolts
Labels	one of four rhythm classes per recording, assigned by experts: `N`, `A`, `O` or `~`, explained below

The four labels. The heart has its own pacemaker, the sinus node, which fires regularly and sets off every normal beat. The labels say how far the rhythm departs from that (Figure 2):

- **`N`, normal sinus rhythm.** The sinus node drives every beat: each beat has a P wave, then a QRS, then a T wave, and the RR intervals change only gradually, for example with breathing. On the plot the beats are evenly spaced and look alike.
- **`A`, atrial fibrillation (AF).** The atria quiver instead of contracting, so there are no P waves, only a wavy baseline, and the ventricles are triggered at random moments, so the RR intervals change abruptly from one beat to the next: the rhythm is *irregularly irregular*. The QRS itself looks normal, because the ventricles still contract normally. AF is the most common sustained arrhythmia and raises the risk of stroke, which is why detecting it from a wearable matters.

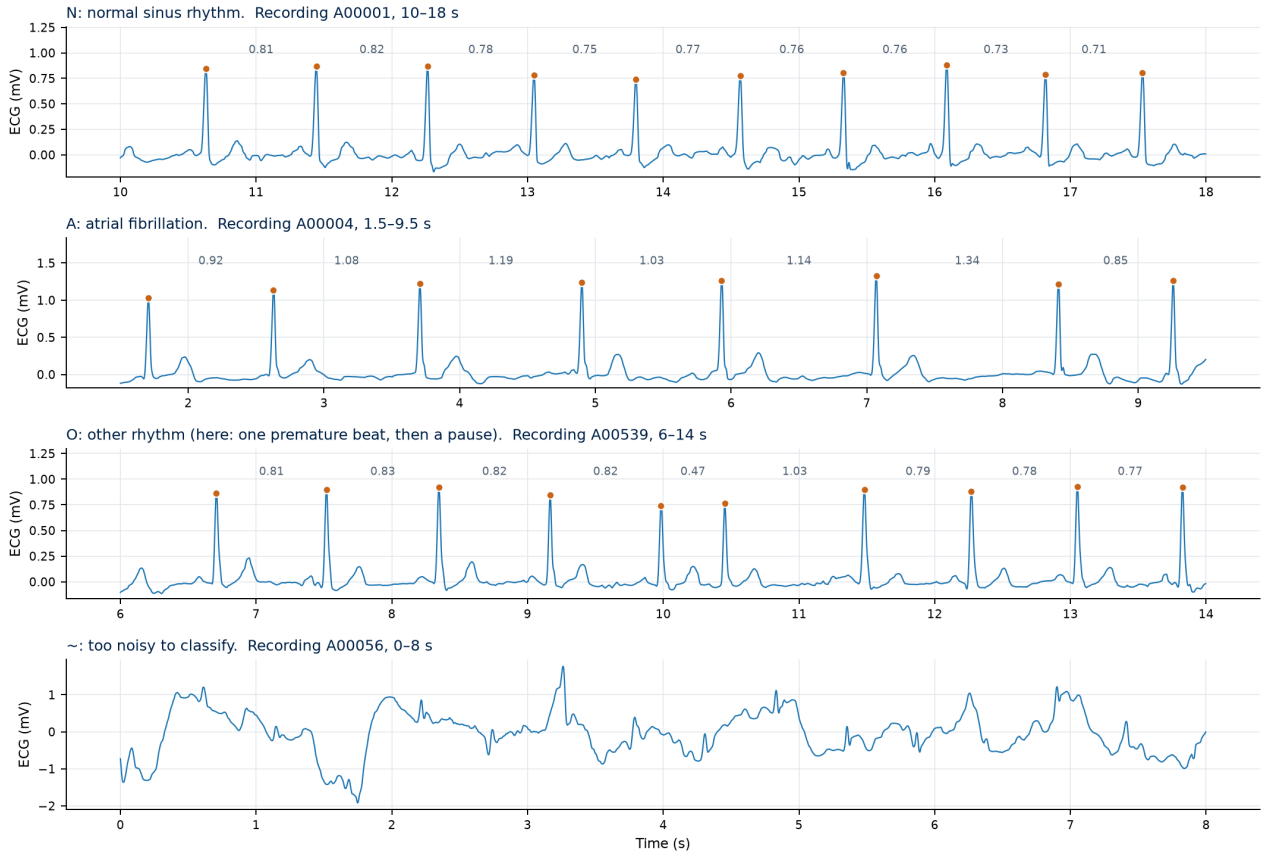


Figure 2: One recording per label, 8 seconds each. Dots mark the beats and the numbers above them are the RR intervals in seconds. `N`: evenly spaced beats with P waves. `A`: intervals that change from beat to beat, and no P waves. `O`: a regular rhythm with one early beat followed by a pause. `~`: no beat can be recognised.

- **`O`, other rhythm.** Everything that is neither normal nor AF: a premature beat that comes early and is followed by a pause (as in the figure), runs of fast beats, a slow rhythm, or blocked conduction. It is a mixed group with no single appearance, which makes it the hardest class in Section 5.
- **`~`, too noisy to classify.** The signal is so corrupted that no rhythm can be read from it.

A label describes the rhythm of the *whole* recording. It says nothing about the quality of the signal (except `~`), and it does not say that every beat is normal: an `N` recording can contain an artefact or a single premature beat.

Task 1 (your code). The notebook shows six excerpts from `N` recordings, each with ten consecutive wave points of two beats marked as numbered dots. The first is labelled as the example (P Q R S T P Q R S T). The other five start at a different wave of the cycle, so each has to be read from the shape of the signal, not from the position. *Deliver:* the ten letters of

each of the five diagrams, in order, with the notebook's check passed; it says which dots are wrong and never prints the answer.

Carry forward. The R peak is the landmark and the RR interval is the first feature. An `N` label does not promise a clean signal: drift, bursts and inversion are in this dataset, and every step that follows has to survive them.

2 What do people measure from an ECG? The rhythm

Notebook: Section 2

In one sentence. The rhythm of the heart is described by the times of its beats: the RR intervals between consecutive R peaks, their average, which gives the heart rate, and how much they change from beat to beat, which is heart rate variability (HRV). Everything in this section is arithmetic on the beat times of Section 1, and the two numbers it defines, heart rate and RMSSD, are the ones you will compute, check and chase through the rest of the lab.

From beats to intervals. From K beats at sample indices $s_1, \ldots, s_K$ and sampling rate f_s:

$$t_k = s_k / f_s \qquad \text{beat times (s)}$$
$$RR_k = t_{k+1} - t_k \qquad \text{RR intervals (s), } K-1 \text{ of them.}$$

Plotting RR_k against time (Figure 3, top) is the quickest way to see a rhythm. The `N` recording is a nearly flat line that drifts slowly. The `A` recording jumps from one interval to the next. The `O` recording is flat except for one short interval followed by one long one: the premature beat and its pause.

Heart rate. Two versions are in use:

$$HR_k = 60 / RR_k \qquad \text{instantaneous heart rate (bpm)}$$
$$HR_{\text{period}} = 60\,(K-1)/(t_K - t_1) = 60/\overline{RR} \qquad \text{rate over the period (bpm).}$$

The period rate is the number a wearable shows. All three recordings in the figure are in the resting range, 63 to 79 bpm, and the bottom-left panel shows the same across the dataset: the four labels overlap heavily. Heart rate alone does not tell the rhythms apart.

Heart rate variability. With the $N = K - 1$ intervals in milliseconds, the two most used time-domain features [4] are

$$\text{SDRR} = \sqrt{\frac{1}{N-1}\sum_{k=1}^{N}\left(RR_k - \overline{RR}\right)^2}, \qquad \text{RMSSD} = \sqrt{\frac{1}{N-1}\sum_{k=1}^{N-1}\left(RR_{k+1} - RR_k\right)^2}.$$

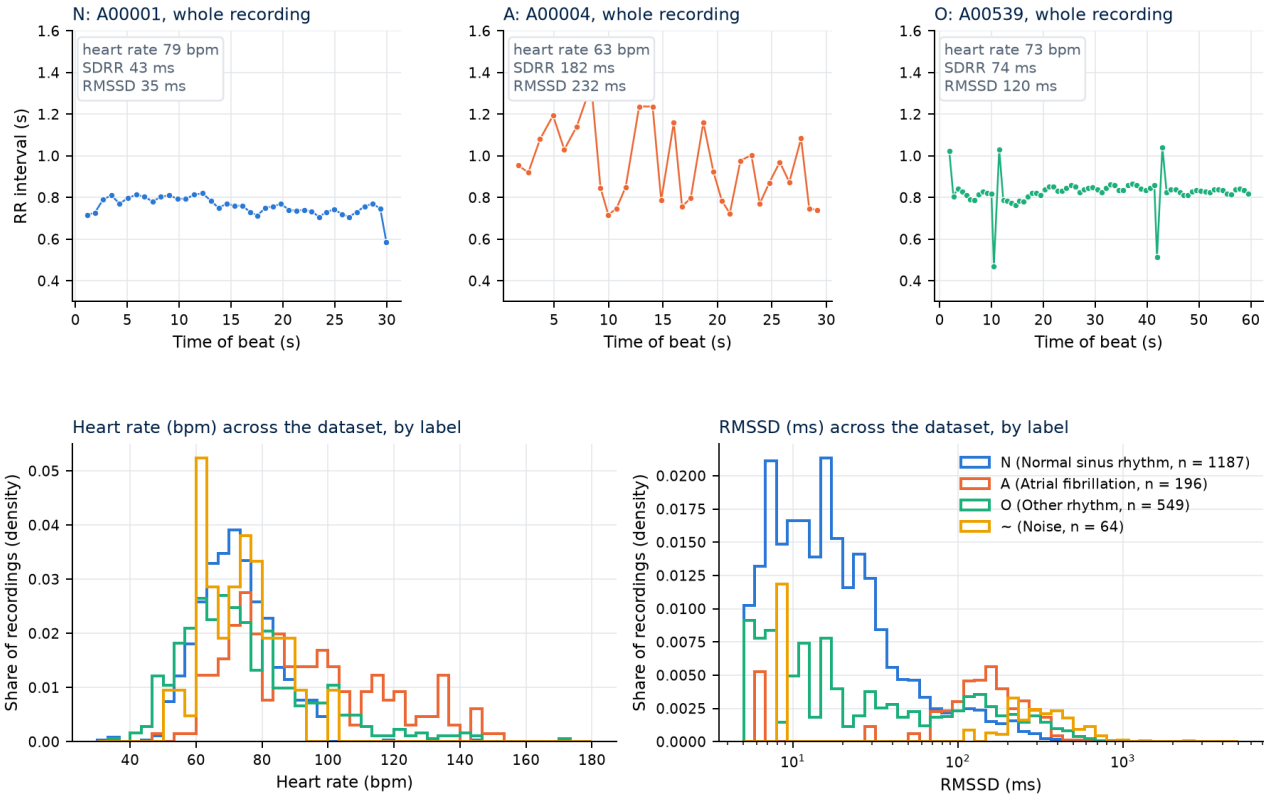


Figure 3: Top: the RR intervals of the `N`, `A` and `O` recordings of Figure 2, over the whole recording, with the heart rate, SDRR and RMSSD calculated from them. Bottom: heart rate and RMSSD (log axis) for every recording in the dataset, by label. Medians: `N` 71 bpm and 49 ms; `A` 89 bpm and 188 ms; `O` 71 bpm and 181 ms; `~` 73 bpm and 394 ms. Beats found by NeuroKit2.

SDRR is the spread of the intervals around their mean; RMSSD is the typical change from one interval to the next. They answer different questions. A slow drift of the rate, as in the `N` recording, gives some SDRR (43 ms) and less RMSSD (35 ms). Abrupt changes, as in atrial fibrillation, give a lot of both (182 and 232 ms). One premature beat, as in the `O` recording, moves SDRR somewhat (74 ms) and RMSSD a lot (120 ms), because the short interval and the long one after it are two large successive differences. In the bottom-right panel RMSSD separates `N` from `A` well, with medians of 49 and 188 ms. `O` overlaps both, which is why it is the hardest class in Section 5, and `~` is largest of all, because wrongly placed beats look exactly like an irregular rhythm. Two cautions: call SDRR *SDNN* only when the intervals are confirmed to be normal-to-normal, and treat values from recordings this short as descriptive, not as a clinical assessment.

More of the same. NeuroKit2's `hrv_time` returns 25 time-domain HRV features, all variations on the ideas above (Table 1). Section 5 uses all of them, and they carry most of the classifier's performance.

What it measures	Features (NeuroKit2 names)
The typical interval, i.e. the rate	MeanNN, MedianNN, MinNN, MaxNN
Spread of the intervals	SDNN (our SDRR), CVNN, MadNN, MCVNN, IQRNN, Prc20NN, Prc80NN; SDANN1/2/5 and SDNNI1/2/5 over 1, 2 and 5-minute segments, undefined on recordings this short
Change from one interval to the next	RMSSD, SDSD, CVSD, pNN50 and pNN20 (share of successive differences above 50 and 20 ms), SDRMSSD
Shape of the RR histogram	HTI, TINN

Table 1: The 25 time-domain HRV features of `hrv_time`, grouped by what they measure (the `HRV_` prefix of each name is dropped).

Task 2 (your code). NeuroKit2 finds the beats of a recording, and the RR sequence is the differences between consecutive beat times. Implement `rhythm_features`, which turns an RR sequence into the heart rate, SDRR and RMSSD using the formulas above. The notebook runs it on the three recordings of Figure 3 and checks every value against NeuroKit2's own `hrv_time` on the same beats. *Deliver:* the function with the check passed, and the three-row table it prints.

Specification: `rhythm_features(rr_s)`

Given `rr_s`, a 1-D NumPy array of RR intervals in seconds, in time order: about 40 values for a 30 s recording, typically between 0.6 and 1.2 s, built from NeuroKit2's beats.

You may use NumPy only: the mean, the sample standard deviation, differences between neighbouring values, the square root.

Return a tuple of three floats `(heart_rate_bpm, sdrr_ms, rmssd_ms)`: $60/\overline{RR}$; the sample standard deviation of the intervals in ms; the square root of the mean of the squared successive differences in ms, squared before averaging.

Checked by NeuroKit2's `hrv_time` on the same beats, for three recordings, to six significant figures.

Carry forward. Heart rate is $60/\overline{RR}$; RMSSD is the root mean square of the successive differences, and you have coded both. Both need the beat times, and RMSSD needs every one of them to be right, which is where Section 3 starts.

3 Can we extract them ourselves? The recipe

Notebook: 3 (Task 3)

In one sentence. The recipe is: find the beats, then do arithmetic on their times. Finding the beats has two parts. A *peak finder* turns the signal into candidate beat times by looking for the fast, tall QRS. *Median imputation* then uses the rhythm itself, the median RR interval, to drop beats that come too early and to fill in the beats that were missed: imputation is the general name for filling missing values from the rest of the data, and here the value used is the median interval. The arithmetic is the few lines you wrote in Section 2; the two parts are the work, and Section 4 builds them one step at a time.

Part 1: the peak finder. The QRS is the fastest and usually the tallest event of each beat (Section 1), so a peak finder emphasises fast changes, thresholds the result, and places one beat in each region above the threshold, at the R peak. Section 4 builds it in three steps (Table 2): filter out drift and fast noise; differentiate, square and integrate, so that each QRS gives one bump; locate the R peak inside each bump. On the recording in Figure 4 it finds 34 beats: 33 real ones, an artefact at 1.0 s, and none at 25.8 s, where the QRS was too small to cross the threshold.

Task 3 (your code): detector v1. The simplest peak finder is the threshold alone, and it is the first version of your detector. Write `naive_beats`, which takes an ECG signal and a threshold in millivolts and returns the sample index of each beat: one beat per region where the signal is above the threshold, at the region's highest sample. The notebook checks it on an invented signal and against a reference on your recording, shows your beats with the threshold next to NeuroKit2's, and turns them into heart rate and RMSSD with your `rhythm_features`: your first features extracted end to end. *Deliver:* the function with both checks passed, and the plot.

Specification: `naive_beats(signal, threshold)`

Given `signal`, a 1-D NumPy array of one ECG lead in mV, one value per sample at $f_s = 300$ Hz (9,000 samples for 30 s); `threshold`, one number in mV. The signal rises above the threshold at every QRS complex, and sometimes at a tall T wave or an artefact; each crossing is a run of consecutive samples above it, a *region*.

You may use NumPy only: comparisons, padding, differences between neighbouring values, the positions of non-zero values, slicing, and the position of a maximum.

Return a 1-D integer array of sample indices, one per region, at the region's highest

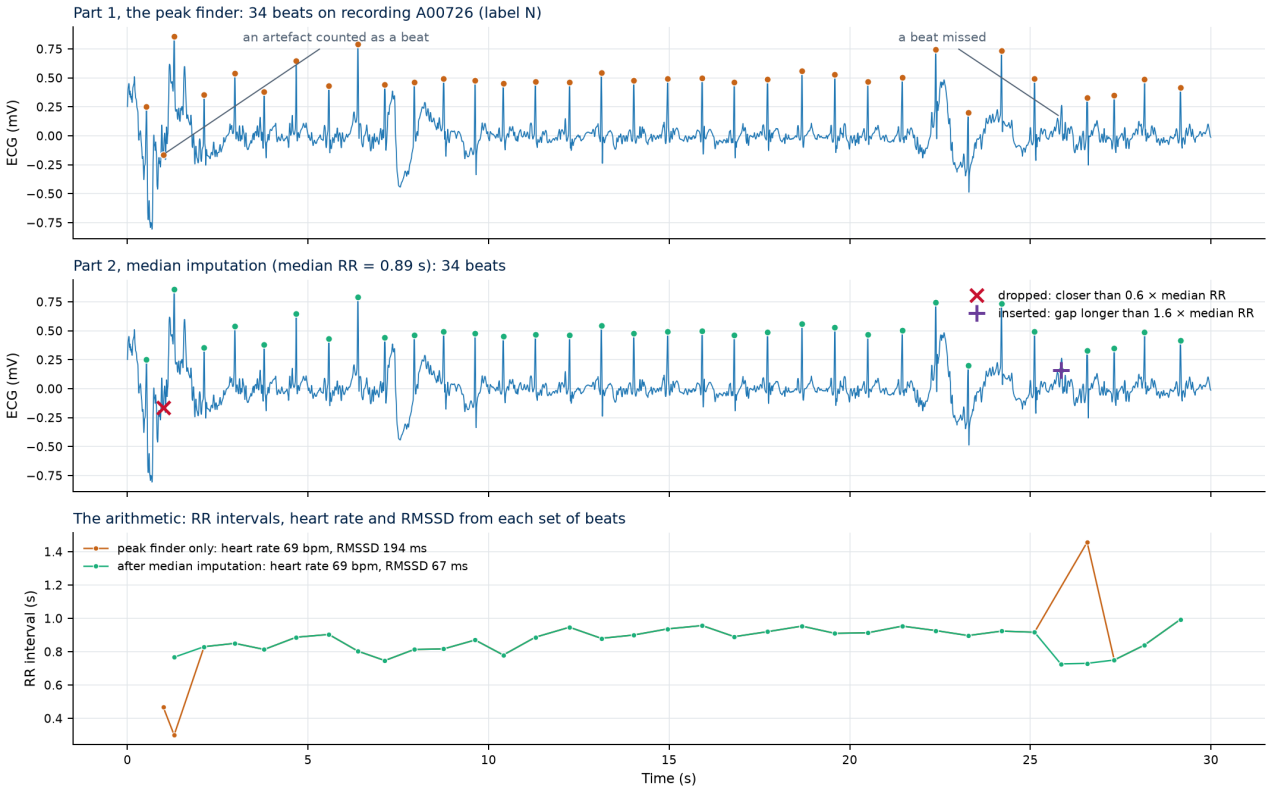


Figure 4: The recipe on an `N` recording. Top: the peak finder's beats; it counts an artefact at 1.0 s and misses a beat at 25.8 s. Middle: the median imputation, with a median RR of 0.89 s, drops the first (closer than 0.6× the median RR to the previous beat) and inserts the second (a gap longer than 1.6× the median RR). Bottom: the RR intervals and the numbers from each set of beats. The heart rate does not change; RMSSD falls from 194 to 67 ms.

sample, in increasing order; empty if nothing crosses. Time in seconds = index / f_s.

Checked by the invented signal `[0, 1, 3, 1, 0, 0, 2, 5, 2, 0, 1, 0]` with threshold 1.5, which must give `[2, 7]`; and a reference implementation on your recording.

Part 2: median imputation. A peak finder has no idea of rhythm: it judges each region on its own. Median imputation adds that idea. The median RR interval is the recording's typical interval, and a few wrong beats barely move it. Going through the beats in time order, a beat closer than 0.6× the median RR to the last kept beat is dropped as an extra, and a gap longer than 1.6× the median RR is split evenly into round(gap / median RR) intervals, inserting beats. If the median rate is outside 30 bpm to 200 bpm, most beats are wrong and the median cannot be trusted, so nothing is changed. On the recording in the figure it removes the artefact and fills the gap: RMSSD falls from 194 to 67 ms while the heart rate stays at 69 bpm. *Price:* inserted beats are invented and evenly spaced, so they pull RMSSD down; a genuine premature beat, as in Section 1's `O` example, is "corrected" away; and in atrial fibrillation the irregularity is the

physiology, not noise. The rule is a teaching version of two established ideas: QRS detectors search back for a missed beat when no beat has been found within about one and a half times the average RR interval [5, 6], and HRV editing identifies extra and missed beats as intervals that deviate from the local distribution and corrects them by merging or splitting intervals [7]. The current standard, used by Kubios and by NeuroKit2's `signal_fixpeaks`, classifies beats with time-varying thresholds on the successive differences [8]. The rule used here is the simplest member of that family, and that is why it was chosen for teaching: it fits in a few lines, and every decision it makes can be read off the RR plot. The factors 0.6 and 1.6 are teaching choices, not taken from any of these papers.

Part	Notebook function (step)	What it does
Peak finder	`naive_beats` (Task 3); in the benchmark `find_regions`, `highest_in_regions`	threshold the signal; one beat per region above it
Peak finder	`bandpass` (4A)	raw ECG → filtered ECG
Peak finder	`differentiate`, `envelope` (4B)	filtered ECG → one bump per beat
Peak finder	`locate` (4C)	regions → beats on the ECG
Median imputation	`correct` (4D)	beats → corrected beats
Arithmetic	`rhythm_features` (Task 2)	RR intervals → heart rate, SDRR, RMSSD

Table 2: The recipe, as the notebook's functions. The regions are found on the last signal that was calculated, and the parts always run in this order.

Carry forward. The recipe is peak finder, median imputation, arithmetic. The arithmetic is done (Task 2) and detector v1 is yours (Task 3). Section 4 builds detector v2 from it, the two parts one step at a time and scores every step against a reference, so that you can see what each step buys and what it costs.

4 Finding the beats: detector v2, which denoises, seeks peaks and imputes

Notebook: 4, 4A–4E

In one sentence. The goal of this section is to get as close as possible to NeuroKit2, an established ECG library, with a detector you build yourself, and so to understand what the industry has been doing for decades to get features out of ECG signals. Detector v2 grows out of your detector v1 in four steps, three for the peak finder and one for median imputation, in the style of the QRS detector of Pan and Tompkins [5], and every step is scored against NeuroKit2 across all 1,996 recordings, so that you see what it buys and what it costs. Each step is introduced by the *problem* the previous detector still has, the *idea* that addresses it, and the *price* it pays. The figures follow recording `A00726` from Section 3 through every step. The settings are teaching choices for 300 Hz data, not a validated clinical detector.

4.1 A reference and a score

Notebook: 4: the benchmark, the NeuroKit2 reference, detector v1

The reference. NeuroKit2 [1] is an open-source library used across research and industry to turn an ECG into beats and features, and its default pipeline (`ecg_clean`, then `ecg_peaks`) belongs to the same family as the recipe of Section 3: filter, emphasise the fast QRS, threshold, place the beat. It is the *NeuroKit2 reference* for this section, as the notebook calls it: its beats for every recording, with no correction, and the heart rate and RMSSD from them (the circles in Figure 5). Every step you add is a step the library also takes, so closing the gap to it is how you find out what sits inside a library call, and where the practice has choices. Two things to keep in mind. The dataset labels rhythms, not beats, so there is no ground truth here: the reference is an established pipeline, not the truth, and a gap to it means *different*. And both detectors can make the same mistake, which no score can see.

The score. In the notebook, each step is one short function that you write from its description (Tasks 4A to 4D), with its setting next to it and a self-check against a reference; a detector is the set of functions so far, kept in a dictionary: `steps_naive`, then `steps_4a = {**steps_naive, "filter": bandpass}`, and so on. `bench.score(name, steps)` runs a detector on every recording, prints three scores, enters it on a leaderboard and plots it next to the reference:

- **beat agreement**: beats paired with a reference beat within ±100 ms, as 2 × matched/(ours + reference); 1 when every beat pairs;

- **heart rate error**: |ours − reference| per recording, summarised by the MAE (mean absolute error, the typical error) and the P95 (the error only the worst 5% of recordings exceed);
- **RMSSD error**: the same, in ms.

Detector v1 on the benchmark. Your Task 3 rule, with one change: amplitude differs between people, electrodes and recordings, so the threshold is relative, 20% of the recording's maximum in its middle half, rather than a value in millivolts. Each region above the threshold gives one beat, at its highest point. On `A00001` this finds most beats; on `A00726` it finds 54 beats where there are 34, because P and T waves and a baseline swing also cross the threshold (Figure 5, and the top panel of Figure 6). Every later step keeps this threshold rule and changes one thing before it.

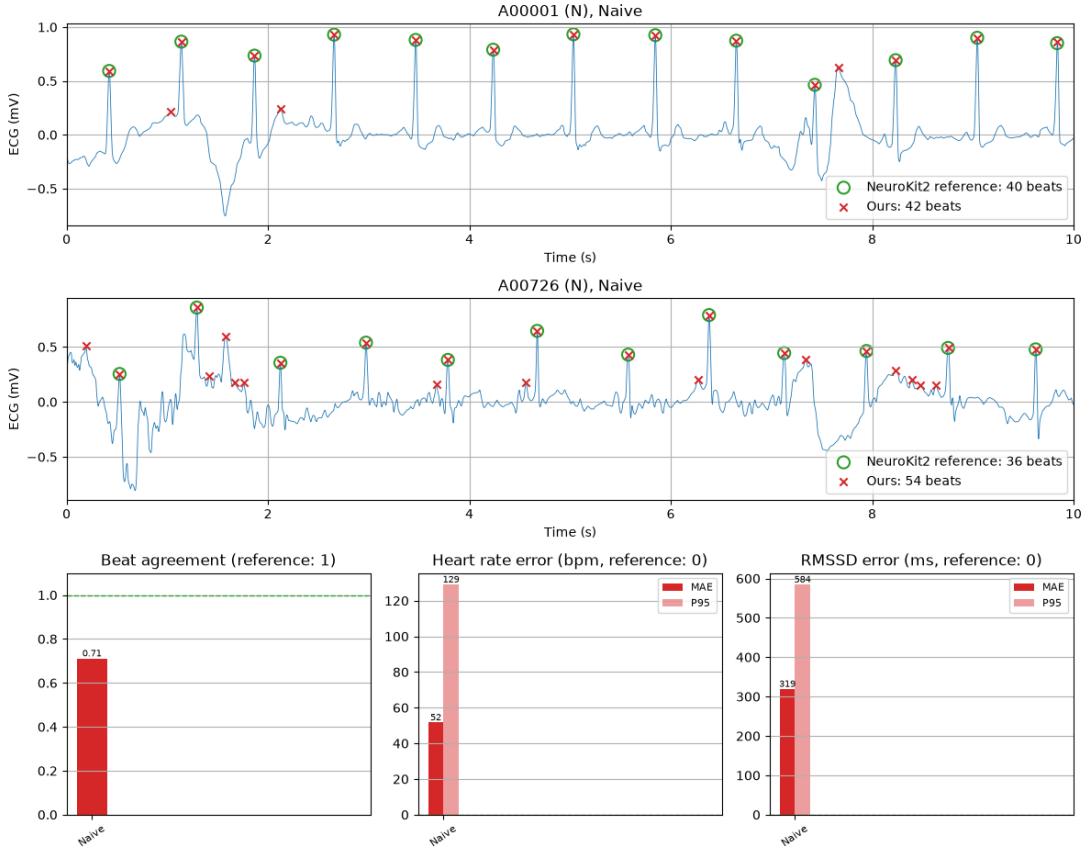


Figure 5: The naive detector (crosses) and the NeuroKit2 reference (circles) on `A00001` and `A00726`, and its scores across the dataset.

4.2 Peak finder, step 1: denoise with a band-pass filter

Notebook: 4A

Problem: the baseline drifts and fast noise rides on the signal; both can cross a threshold. *Idea:* keep the band where the ECG waves live, 0.5 Hz to 40 Hz, with a Butterworth filter applied forwards and backwards so that peaks are not shifted. *Price:* the band is an assumption; real features outside it are distorted, and a corrupted segment stays corrupted.

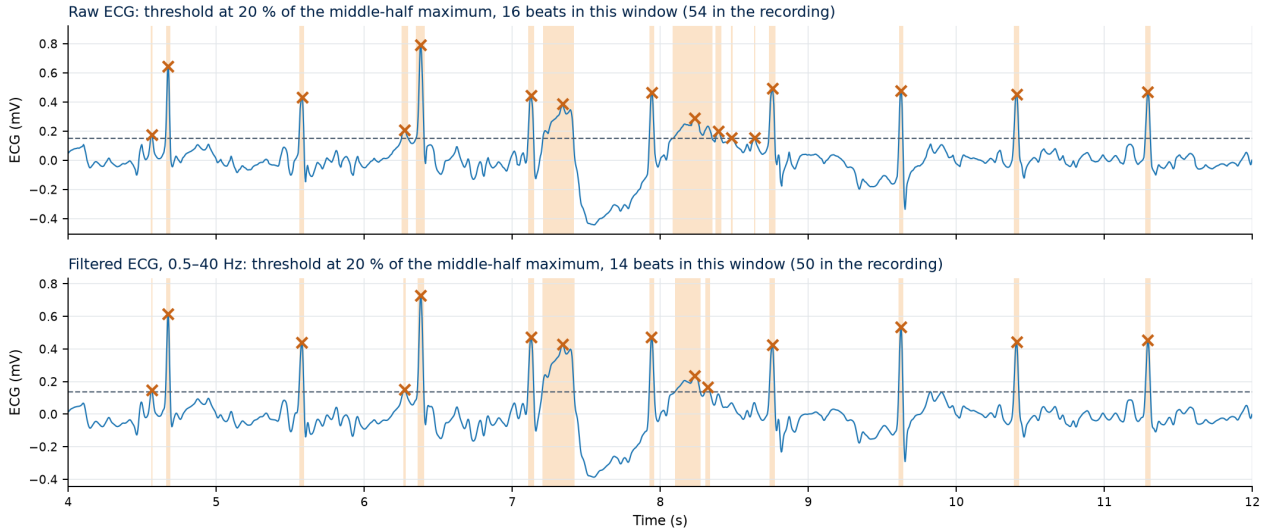


Figure 6: Step 1 on `A00726`, 4 to 12 s: the naive threshold rule on the raw ECG (top) and on the filtered ECG (bottom), with the regions above the threshold shaded and the beat each region gives. The baseline is flatter and two P-wave crossings are gone, but the swing at 7 s to 8 s still crosses the threshold.

What it bought. Almost nothing: beat agreement 0.71 to 0.72, heart rate MAE 51.7 to 52.6 bpm. The threshold is still a test of *height*, and the things that fool it, T waves, swings and inverted QRS complexes, are inside the band.

Task 4A (your code). Write `bandpass`: design the filter with `butter` (order 4, the band 0.5 Hz to 40 Hz, `fs=FS`, second-order sections) and apply it with `sosfiltfilt`. *Deliver:* the function with the check passed, and its leaderboard line.

Specification: `bandpass(x)`

Given `x`, a 1-D array, the whole raw recording in mV at 300 Hz, with baseline drift and fast noise on it; the settings `low_hz` and `high_hz` (0.5 and 40) above the function.

You may use SciPy's `butter` and `sosfiltfilt`, imported at the top of the cell; their documentation gives the arguments.

Return a float array of the same length: the ECG through a fourth-order Butterworth band-pass between the two cutoffs, as second-order sections, applied forwards and then backwards so that no peak is shifted in time.

Checked by a reference implementation on your recording.

4.3 Peak finder, step 2: differentiate, square, integrate

Notebook: 4B

Problem: height is a poor clue; a tall T wave can be as high as the R peak, and some QRS complexes point down. *Idea:* the QRS is different because it is *fast* (Figure 7). Differentiate, $d[n] = x[n] - x[n-1]$, to measure speed: large at the sharp QRS edges, small on slow waves. Square, so that both signs count and large values grow much more than small ones. Integrate with a centred moving average about one QRS wide (120 ms, 37 samples), merging the spikes of one beat into one bump. Only together do the three give the threshold one bump per beat, so they are one step. *Price:* a strong noise spike is emphasised just as much as a QRS, and the bump is smeared in time.

What it bought. Most of the gap: beat agreement 0.72 to 0.92, heart rate MAE 52.6 to 6.4 bpm. The RMSSD MAE also falls, 288 to 218 ms, but its P95 rises, 516 to 648 ms: on a few recordings a strong artefact now sets the threshold for the whole recording.

Task 4B (your code). Write `differentiate` (the difference of neighbouring samples, with a leading zero so the length is unchanged) and `envelope` (square, then a centred moving average over the 120 ms window, rounded to an odd number of samples). *Deliver:* both functions with the checks passed, and the leaderboard line.

Specification: `differentiate(x)` and `envelope(d)`

Given `x`, the filtered ECG, a 1-D float array in mV; then `d`, the difference signal, with positive and negative spikes at every QRS; the setting `integration_s` (0.12) above the function.

You may use NumPy only (for the envelope, a convolution with a window of equal weights, or cumulative sums).

Return `differentiate`: `d`, same length as `x`, with $d[n] = x[n] - x[n-1]$ for $n \geq 1$ and $d[0] = 0$. `envelope`: a non-negative array of the same length, at each sample the mean of d^2 over a window of `width` samples centred on it, where `width` is `integration_s` $\times f_s$ rounded to an integer and made odd (37 samples at 120 ms); beyond the edges the signal



Figure 7: Step 2 on the same excerpt: filtered ECG, difference, squared difference, and the envelope with its threshold and regions. The swing that fooled step 1 is slow, so it almost vanishes in the difference; the envelope has one bump per beat, and the threshold now finds exactly the nine beats in the window.

counts as zero.

Checked by the invented signal `[0, 1, 3, 1, 0]`, which must differentiate to `[0, 1, 2, -2, -1]`; and a reference envelope on your recording.

4.4 Peak finder, step 3: locate the R peak on the ECG

Notebook: 4C

Problem: the envelope says roughly *when* a beat happens, but its maximum is smeared and sits after the R peak, and RR intervals are differences of these times. Occasionally one beat still gives two regions. *Idea:* in each candidate region, place the beat at the maximum of the filtered ECG (the minimum for an inverted recording, chosen only after inspecting the waveform), and keep no two beats closer than 250 ms, strongest first (Figure 8). *Price:* this rule can reject true fast beats or keep strong artefacts.

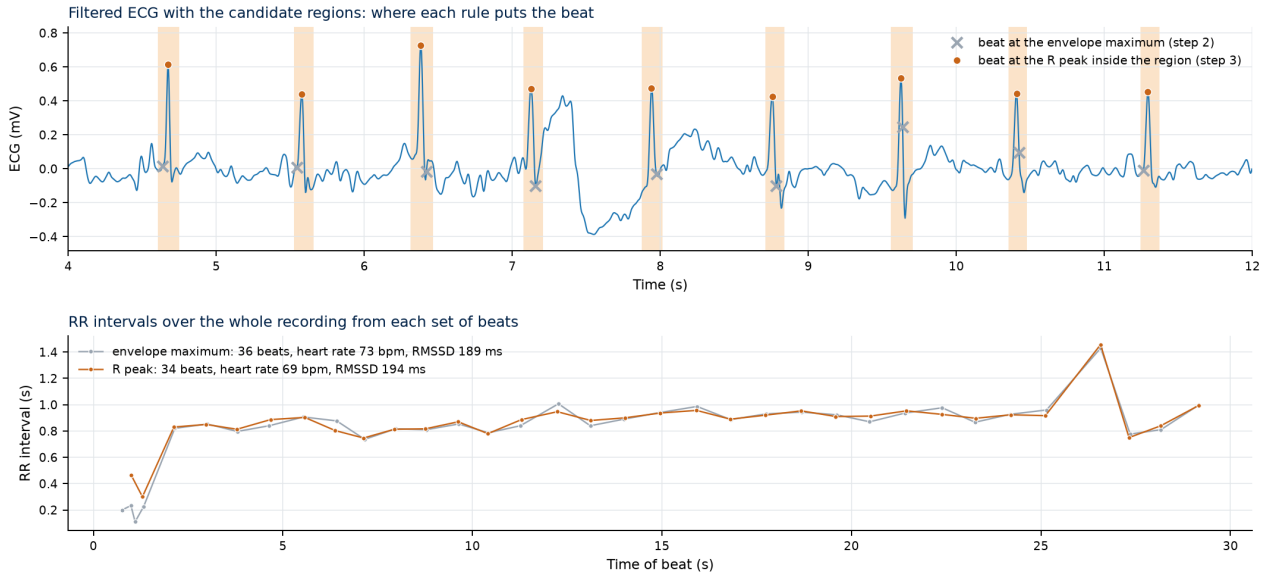


Figure 8: Step 3 on `A00726`. Top: in each region, the beat at the envelope maximum (step 2) sits about 40 ms after the R peak, where step 3 puts it. Bottom: the RR intervals from each set of beats over the whole recording. The grey line zigzags where the orange one is smooth; the two large spikes, the artefact at 1 s and the missed beat at 26 s, are left for median imputation.

What it bought. Beat agreement hardly moves (0.92 to 0.91: the envelope maximum was already within the 100 ms pairing tolerance), but the heart rate MAE falls from 6.4 to 5.1 bpm and the RMSSD MAE from 218 to 194 ms, because the intervals are now measured between R peaks and not between smeared maxima.

Task 4C (your code). Write `locate`: one candidate per region, the highest sample of the filtered ECG (the lowest for negative polarity); then go through the candidates strongest first and keep one only if it is at least 250 ms from every beat already kept. *Deliver:* the function with the checks passed, and the leaderboard line.

Specification: `locate(ecg, starts, stops)`

Given `ecg`, the filtered ECG in mV; `starts` and `stops`, integer arrays of equal length in time order, region k being `ecg[starts[k]:stops[k]]`, the regions disjoint, each meant to hold one beat but occasionally two regions on one beat. Inside the function `score` is defined: `ecg` for positive polarity, `-ecg` for negative, so the beat is always the highest sample of `score`. The settings `polarity` and `min_separation_s` (0.25) are above the function.

You may use `highest_in_regions(signal, starts, stops)`, supplied in the

detector v1 cell, which returns the highest sample of each region; Python's sorting; NumPy.

Return a 1-D integer array of beat indices in time order: one candidate per region, with every candidate closer than `min_separation_s` $\times f_s$ samples to a *stronger* kept candidate removed. Going through the candidates strongest first guarantees that the stronger of two close candidates is the one kept.

Checked by an invented signal with peaks of height 5, 6 and 3 at samples 100, 150 and 500, which must give `[150, 500]`; and a reference implementation on your recording.

4.5 Median imputation: correct the beats with the median RR

Notebook: 4D

This is part 2 of the recipe (Section 3): the simplest member of a family that descends from the search-back rule of QRS detectors [5, 6] and from interval-based HRV editing [7, 8], chosen for its simplicity. *Problem:* a few beats are still wrong: an artefact taken for a beat, or a small QRS that never crossed the threshold. Heart rate barely notices; RMSSD does, because one missed beat creates two large successive differences in a row. *Idea:* use the rhythm itself. In time order, a beat closer than 0.6× the median RR to the last kept beat is dropped; a gap longer than 1.6× the median RR is split evenly, inserting beats; nothing is changed if the median rate is outside 30 bpm to 200 bpm. Figure 4 shows it on `A00726`: the artefact dropped, the missed beat inserted, RMSSD 194 to 67 ms. *Price:* inserted beats are invented and evenly spaced, so they pull RMSSD down; a genuine premature beat is "corrected" away; and in atrial fibrillation the irregularity *is* the physiology. The reference is not corrected, so where it has an error of its own, correcting ours moves us away from it.

What it bought. The RMSSD MAE falls from 194 to 146 ms and its P95 from 604 to 393 ms; the heart rate P95 from 32 to 28.5 bpm. Beat agreement slips from 0.91 to 0.90: where the reference has a missed or extra beat of its own, our corrected beats no longer pair with it.

Task 4D (your code). Write `correct`: the median RR of the beats, then the walk in time order with its two rules, drop when closer than 0.6 × the median, fill when further than 1.6 ×; the rate guard is supplied. *Deliver:* the function with the checks passed, and the leaderboard line.

Specification: `correct(beats)`

Given `beats`, a 1-D integer array of sample indices in time order from detector 4C, usually 30 to 60 per recording, occasionally with an extra one (an artefact) or a missing one (a small QRS); the settings `correct_short` and `correct_long` (0.6 and 1.6) above the function. The rate guard, the walk through the beats and the insertion of evenly spaced beats are supplied; you write the four quantities the rule needs.

You may use NumPy only.

Return a 1-D integer array in time order, the beats with every extra one dropped and every gap filled. `median_rr` is the median RR interval in samples; for each beat, `gap` (supplied) is its distance to the last kept beat; `too_close` is whether the gap is shorter than `correct_short` × `median_rr`; `too_far` is whether it is longer than `correct_long` × `median_rr`; `k` is the number of intervals the gap should hold, `gap / median_rr` rounded to an integer.

Checked by the invented beats `[0, 300, 600, 900, 1500, 1800, 1850, 2100]` (median RR 300), which must give `[0, 300, 600, 900, 1200, 1500, 1800, 2100]`; and a reference implementation on `A00726`.

4.6 How close did we get?

Notebook: 4E, take one out

Table 3 and Figure 9 give the gap after each step, and the gap that reopens when one step is removed from the full detector.

Detector	Beat agreement	HR error (bpm) MAE / P95	RMSSD error (ms) MAE / P95
Added one at a time			
Naive	0.71	51.7 / 129	319 / 584
+ filter (4A)	0.72	52.6 / 147	288 / 516
+ diff., square, integrate (4B)	0.92	6.4 / 33	218 / 648
+ locate (4C)	0.91	5.1 / 32	194 / 604
+ correct (4D)	0.90	5.0 / 28.5	146 / 393
Removed from the full detector			
without filter	0.90	4.9 / 28.5	152 / 397
without diff., square, integrate	0.81	20.0 / 89.3	132 / 408
without locate	0.92	5.7 / 30.5	176 / 417
without correct	0.91	5.1 / 31.7	194 / 604

Table 3: The leaderboard across 1,996 recordings: the gap to the reference after each step, and with one step taken out of the full detector.

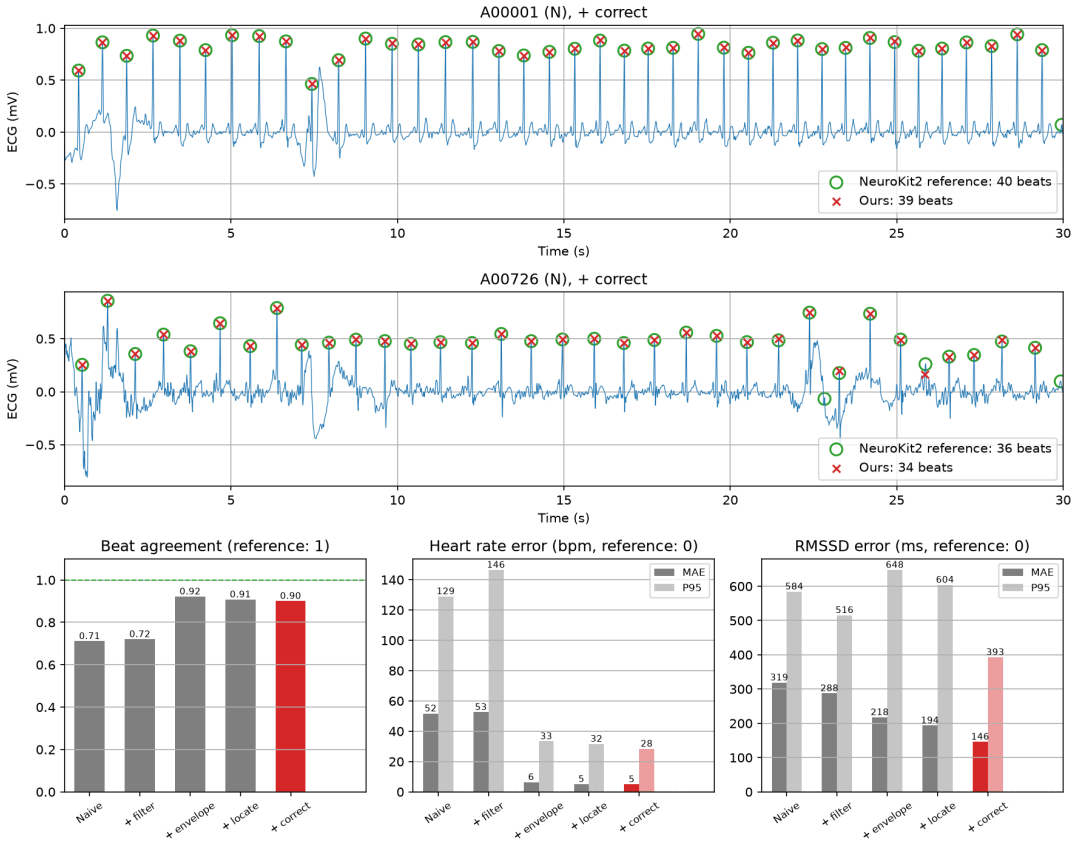


Figure 9: The full detector on `A00001` and `A00726`, and the leaderboard.

An MAE far above the median error says that a few recordings carry most of it. The notebook's 4E cell plots each recording's heart rate and RMSSD against the reference's (an identity plot and a Bland–Altman plot of reference − ours, with the bias and the 95% limits of agreement), with the 53 recordings where either detector's median rate is implausible excluded as detection failures. All of this is *agreement, not accuracy.*

The notebook's `bench.compare` draws this comparison for your detector; nothing is entered on the leaderboard.

Take one out. Adding steps one at a time shows what each adds *on top of the ones before it.* The reverse test removes one step at a time from the full detector and scores it again; the notebook's `bench.take_one_out` does this and plots the gaps next to the full detector's (the lower block of Table 3). Removing the filter costs little, as adding it gained little: the envelope step already removes slow drift, and this dataset has little fast noise. Removing differentiate, square and integrate costs most, although adding the filter before it gained almost nothing: a step's value depends on which other steps are present.

Carry forward. You have now extracted two basic features, heart rate and RMSSD, with a detector you built in four steps, and it gets close to an established library on most recordings: that is what the industry's feature extraction does, and you know why each piece is there. The score cannot tell you which of the two is right; only looking at the beats can, and nobody looks at 2,000 recordings. Section 5 lets the library extract the full set of features for every recording, and puts them to the use they were designed for: classifying the rhythm.

5 What the features are for: classifying the rhythm

Notebook: Section 5

Why classify. A wearable ECG exists to catch what a clinic visit misses. Atrial fibrillation often comes and goes, is often silent, raises the risk of stroke about fivefold [9], and can be treated once it is found. So people record short strips at home, by the million, and no cardiologist can read them all. A classifier that labels each recording `N`, `A`, `O` or `~` is the triage: likely AF goes to a clinician, noise is recorded again, normal is filed. The label is a screening flag, not a diagnosis. The 2017 challenge that produced this dataset was set up for exactly this task, by the makers of such a device [2].

From two features to the full set. In Section 4 you extracted two basic features, heart rate and RMSSD, with a detector you built, and matched what a library does to get them. Here the library does the whole job: for every recording, NeuroKit2 runs the reference pipeline unattended and returns the full set of 54 features, and a RandomForest [10] learns the labels from them. This is what the features are for. Section 2 showed that one rhythm feature already separates `N` from `A` to a useful degree, but `O` overlaps both and `~` is scattered, so no single threshold will do; a classifier combines many features and draws the boundary itself. The route is interpretable, you can see which features the forest relies on (Figure 10), and it learns from about 1,400 labelled recordings. Its price is that every new problem needs new expertise and new features. Lab 2 asks what happens when nobody designs them.

5.1 54 features for one recording

Notebook: 5A

`extract_feature` runs the reference pipeline on a whole recording (`ecg_clean`, then `ecg_peaks`, no correction), locates the P, Q, S and T waves around each beat with `ecg_delineate`, and returns 54 numbers: 8 from R-peak timing (the R amplitude, the RR intervals before, at and after each beat, and their ratios), 20 from wave shape (the times and amplitude differences between the P, Q, R, S and T peaks), the 25 time-domain HRV features

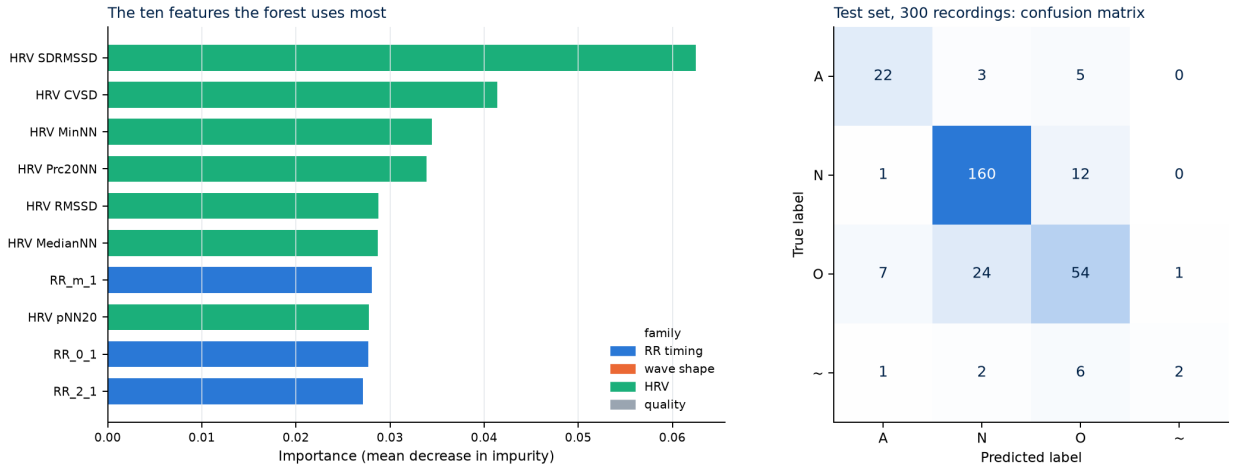


Figure 10: Left: the ten features the forest uses most, coloured by family; all ten are rhythm features. Right: the confusion matrix on the 300 test recordings, true label by row, predicted label by column.

of Table 1, and 1 signal quality index. It runs without visual review: that is what makes 2,000 recordings feasible, and also the risk: nobody checks the beats.

5.2 The feature table and the classifier

Notebook: 5B, Task 5

Extracting features from all 2,000 recordings takes a long time, so the table (`ECG_feature.pkl`) was computed in advance with the same function; rows where extraction failed are all zeros. The recordings are split 70% / 15% / 15% into training (1,400), validation and test sets with `random_state=42`, the same split as Lab 2, so that Lab 2 can compare its networks with this classifier. You then train the classifier (Task 5): the features are standardised with the training set only, and a RandomForest of 100 trees is fitted on them. The classes are unbalanced, so the score to read is the *macro-averaged F1*: the mean of the four classes' F1 scores, so that each class counts equally (Table 4).

Class	Precision	Recall	F1	Support
`N` normal sinus rhythm	0.85	0.92	0.88	173
`A` atrial fibrillation	0.71	0.73	0.72	30
`O` other rhythm	0.70	0.63	0.66	86
`~` too noisy	0.67	0.18	0.29	11
Macro average	0.73	0.62	**0.64**	300

Table 4: The RandomForest on the 300 test recordings. Accuracy is 0.79; the macro F1 of 0.64 (0.68 on the validation set) is the number Lab 2 compares against.

What the forest relies on. Half of the importance sits in the 25 HRV features and another fifth in the 8 RR-timing features; the ten most used are all rhythm features, led by SDRMSSD and CVSD, which measure variability relative to the typical interval. That fits what distinguishes AF on the ECG: the intervals, not the waves. Wave shape carries the remaining third, spread thinly over 20 features, and the quality index is never used.

Reading the confusion matrix. `N` is recognised (160 of 173) and so, mostly, is `A` (22 of 30, the rest called `O`). `O` is the difficulty: 24 of 86 are called `N` and 7 `A`, because it is a mixed group that overlaps both. `~` is almost never recognised (2 of 11): there are too few examples to learn from, about 45 in training. Macro F1 counts that class as much as `N`, which is why the accuracy of 0.79 becomes a macro F1 of 0.64.

Task 5 (your code). Write `train_forest`: a `StandardScaler` fitted on the training set only, and a `RandomForestClassifier` of 100 trees (`random_state=42`) fitted on the scaled training features; and `predict`: scale new features with the same scaler and return the model's predictions. The notebook checks your test-set predictions against a reference, prints the classification reports and plots the ten features the forest uses most. *Deliver:* both functions with the check passed; the test report; and your test macro F1, written down for Lab 2.

Specification: `train_forest(x_train, y_train)` and `predict(scaler, model, x)`

Given `x_train`, a 2-D float array of 1,400 rows (recordings) × 54 columns (features), unscaled, with ranges that differ by orders of magnitude between columns; `y_train`, an integer array of 1,400 class codes (0 to 3). For `predict`: the two fitted objects and `x`, $n \times 54$ unscaled features.

You may use `StandardScaler` and `RandomForestClassifier`, imported at the top of the cell; their documentation gives the arguments.

Return `train_forest`: `(scaler, model)`, a scaler fitted on `x_train` only (each column centred and scaled by the training set's mean and standard deviation) and a forest of 100 trees with `random_state=42` fitted on the scaled `x_train` and `y_train`. `predict`: an integer array of n predicted class codes, from `x` scaled with the same scaler.

Checked by the reference model's predictions on the 300 test recordings, which yours must match exactly.

Carry forward. From a raw strip to a screening label, every step was designed by a person: the filter band, the envelope window, the median-RR rule, the 54 features. The result is interpretable and learns from 1,400 recordings, and it is only as good as the features and the

beats under them. Lab 2 trains networks on the same recordings and the same split, with no designed features, and compares them with your macro F1.

Submitting your work

Save the notebook with your code and plots. Restart the kernel and run all cells to check that your work is reproducible, and keep your test macro F1 from Section 5 for Lab 2.

Acknowledgement. The learning sequence derives from the CWM Wearable Technology practical by Mauricio Villarroel, with detector material credited to Gari Clifford and Mauricio Villarroel [11]. The current 300 Hz implementation is a teaching adaptation, not a numerical reproduction of that detector.

Bibliography

[1] D. Makowski, T. Pham, Z. J. Lau, J. C. Brammer, F. Lespinasse, H. Pham, C. Schölzel, and S. H. A. Chen, "NeuroKit2: A Python toolbox for neurophysiological signal processing," *Behavior Research Methods*, vol. 53, pp. 1689–1696, 2021.

[2] G. D. Clifford, C. Liu, B. Moody, L.-w. H. Lehman, I. Silva, Q. Li, A. E. Johnson, and R. G. Mark, "AF classification from a short single lead ECG recording: the PhysioNet/Computing in Cardiology challenge 2017," in *Computing in Cardiology*, vol. 44, 2017.

[3] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, and H. E. Stanley, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.

[4] Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology, "Heart rate variability: standards of measurement, physiological interpretation and clinical use," *Circulation*, vol. 93, no. 5, pp. 1043–1065, 1996.

[5] J. Pan and W. J. Tompkins, "A real-time QRS detection algorithm," *IEEE Transactions on Biomedical Engineering*, vol. BME-32, no. 3, pp. 230–236, 1985.

[6] P. S. Hamilton and W. J. Tompkins, "Quantitative investigation of QRS detection rules using the MIT/BIH arrhythmia database," *IEEE Transactions on Biomedical Engineering*, vol. BME-33, no. 12, pp. 1157–1165, 1986.

[7] G. G. Berntson, K. S. Quigley, J. F. Jang, and S. T. Boysen, "An approach to artifact identification: application to heart period data," *Psychophysiology*, vol. 27, no. 5, pp. 586–598, 1990.

[8] J. A. Lipponen and M. P. Tarvainen, "A robust algorithm for heart rate variability time series artefact correction using novel beat classification," *Journal of Medical Engineering & Technology*, vol. 43, no. 3, pp. 173–181, 2019.

[9] P. A. Wolf, R. D. Abbott, and W. B. Kannel, "Atrial fibrillation as an independent risk factor for stroke: the Framingham study," *Stroke*, vol. 22, no. 8, pp. 983–988, 1991.

[10] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[11] M. Villarroel, "CWM wearable technology laboratory code." https://github.com/maurovm/cwm_laboratory_code. Revision 5103c09065bf8ea510461e096471456d0267e101.